SLDS Flutter: How It Works & Running Locally

How it works

The repo is one **Dart pub workspace**, declared in the root `pubspec.yaml`, with four members. Melos 7 reads its config from the `melos:` block in that same file; there is no `melos.yaml`.

```
packages/slds_tokens      -> raw design values (spacing, colors as ints, radii...), pure Dart
packages/slds_components  -> Flutter widgets + theme + l10n, built on slds_tokens
widgetbook/               -> preview app for browsing components one at a time
app/                      -> example consumer app (bloc + clean architecture)
```

`widgetbook` and `app` depend on `slds_components` by local **path**, not through pub.dev. When you edit a widget in `packages/slds_components`, both apps pick up the change on hot reload. You never publish or bump versions to test a change.

1. One-time setup (from the repo root)

```
dart pub global activate melos   # installs the melos CLI
dart pub get                     # resolves all 4 packages into one shared lockfile
```

Because it is a pub workspace, a plain `dart pub get` is enough. `melos bootstrap` does the same thing.

2. Generate Widgetbook's use-case index

Run this the first time, and again whenever you add or rename a `@widgetbook.UseCase`:

```
melos run gen
```

3. Run everything at the same time

Use a separate terminal for each:

Terminal	Command	What it does
1	melos run widgetbook	Opens Widgetbook in Chrome
2	cd app && flutter run	Runs the example app (pick a simulator, or add -d chrome)
3	your editor	Edit packages/slds_components/lib/src/widgets/...

When you save a file in the package, press **r** in terminal 1 or 2 to hot reload, and the change shows up in that app. Nothing needs to "run" for the package itself because it is a library.

In Widgetbook, the toolbar lets you switch Theme (light/dark), Locale (en/si/ta), Viewport (device frames), text scale and so on. That is how you check a component on a phone-sized screen without a real emulator.

4. Tests and lint

```
melos run test                                    # every package that has a test/ folder
(cd packages/slds_components && flutter analyze)  # lint, per package
```

Adding a new component

1. Add the widget under `packages/slds_components/lib/src/widgets/`.
2. Export it from `lib/slds_components.dart`.
3. Add a use case in `widgetbook/lib/use_cases/`.
4. Run `melos run gen`, then hot restart Widgetbook.

Things to know

- The workspace needs Dart `^3.11` (Flutter 3.41 or newer).
- There is no Melos script that launches the example app, so start it with `flutter run` from inside `app/`.

- If you run `melos run test` from a script or CI (no interactive terminal), it stops to ask you to pick a package. In that case, run `flutter test` inside each package instead.